

Intent-Based Networking Engine for an ISP Core Network

Miguel Filipe Coelho Mateus
Faculty of Engineering, University of Porto
M.EIC — Networks and Systems Management
up202206944@fe.up.pt

Abstract—Managing the routing, DNS, load-balancing, and traffic-engineering of an ISP core network through manual per-device configuration is error-prone, slow to converge on failures, and leaves no auditable single source of truth for the desired network state. This paper presents an Intent-Based Networking (IBN) engine that takes a single declarative YAML document describing the desired network state and automatically renders, provisions, and continuously reconciles the full infrastructure stack — OSPF/BGP routing via Quagga/FRR, BIND9 authoritative DNS, Nginx load balancing across four web replicas, and Nagios closed-loop monitoring — without any manual device configuration. A second, optional layer extends the engine with a Ryu OpenFlow controller and Open vSwitch running inside router containers, enabling sub-second traffic rerouting between primary and backup paths when congestion is detected. Experimental results demonstrate strict idempotency (zero delta on re-application), correct delta provisioning on intent changes, and path-switch latency well under one second compared to the 10–40 s convergence time of OSPF alone.

I. INTRODUCTION

Network provisioning in modern ISP or enterprise core networks remains a largely manual, error-prone process. Engineers configure routers, DNS servers, and load balancers by hand, often working from scripts, wiki pages, or undocumented tribal knowledge. A single character error in an OSPF area declaration or a DNS zone file can isolate an entire subnet with no automated safety net to detect or recover from the mistake [10].

Beyond correctness, the absence of a single declarative representation of desired network state makes *drift* — the gradual divergence between what was intended and what is actually running — hard to detect and even harder to remediate at scale. Tools that do exist, such as Ansible [3] or Terraform [2], were designed primarily for cloud resource management and require the operator to describe *how* to reach the target state rather than simply declaring *what* the target state is.

Finally, when network events occur — link congestion, BGP session drops, latency spikes — the only available responses are human intervention or waiting for routing protocols to converge. OSPF convergence, for instance, typically takes 10 to 40 seconds [9], which is unacceptable for latency-sensitive services.

This paper presents the design, implementation, and evaluation of an Intent-Based Networking engine built for a simulated ISP core network. The engine accepts a single YAML intent file describing the desired state of routing, DNS,

load-balancing, and monitoring. It then automatically renders all device configuration files, provisions the infrastructure via SSH, monitors its health through a Nagios feedback loop, and — when the optional SDN layer is active — reroutes traffic in under one second using OpenFlow rules pushed via a Ryu controller to Open vSwitch instances running inside each router container.

The key contributions of this work are:

- A *declarative, idempotent* provisioning engine that applies only the configuration delta computed by a planner module, preventing duplicate or conflicting entries.
- A *closed-loop monitoring* integration that translates Nagios health signals and `tc netem` delay measurements into SDN path-activation decisions.
- A *reproducible lab stack* fully described by a Docker Compose file, making the entire ISP topology — three Quagga routers, DNS, load balancer, web replicas, Nagios, Ryu, and the engine itself — deployable with a single command.

II. RELATED WORK

Several existing systems address parts of the network automation problem; none combines declarative intent, protocol-level provisioning, and sub-second SDN remediation in a single open-source engine.

Cisco DNA Center [1] is a commercial IBN platform that provides a graphical intent layer above Cisco hardware. It requires proprietary hardware, is not inspectable at the configuration level, and offers no mechanism for YAML-driven declarative state.

Terraform and Ansible [2], [3] are general-purpose infrastructure-as-code tools widely adopted in cloud environments. Terraform requires the user to describe the desired cloud resource graph, while Ansible requires the operator to write imperative playbooks. Neither was designed to manage network protocol state (OSPF, BGP, DNS zones) directly, and neither offers a built-in reconciliation loop that detects and heals configuration drift at the daemon level.

OpenDaylight and ONOS [4], [5] are production-grade SDN controllers with extensive feature sets and support for multi-vendor environments. Their complexity and operational overhead exceed the requirements of a lab-scale ISP simulation; Ryu [6] provides a lighter Python-native OpenFlow application framework better suited to this context.

Kubernetes [7] provided the conceptual inspiration for the reconciliation loop pattern used in this engine: a control loop continuously compares desired state (the YAML intent) against observed state (running daemons, active flow rules) and applies the minimum delta to bring them into alignment. The IBN engine applies this pattern to network protocol state rather than container lifecycle.

III. SOLUTION

A. Design Principles

The engine is designed around three principles. First, *declarative intent*: the operator describes what the network should look like, not how to configure it. Second, *idempotency*: running the engine against an unchanged intent produces zero changes to any device. Third, *closed-loop autonomy*: the engine continuously observes network health and reacts to congestion events without human intervention.

B. Architecture Overview

Figure 1 shows the overall data flow. The intent YAML is the single source of truth. The engine pipeline — parser, planner, renderer, and provisioner — transforms it into device configuration and applies it to the running infrastructure. A monitor module polls Nagios and inspects `tc qdisc` on router interfaces; when congestion is detected, the SDN client pushes updated OpenFlow rules to Ryu, which installs them into OVS in under one second.

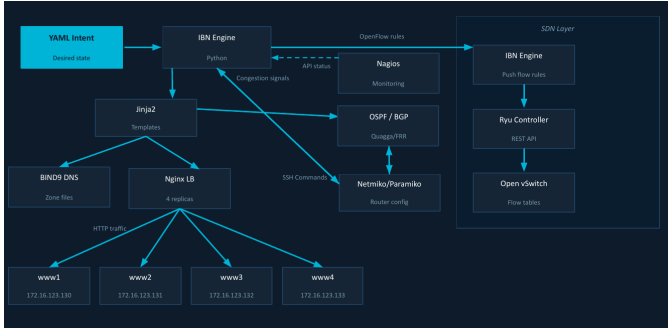


Fig. 1. High-level architectural overview of the IBN Engine. The YAML intent drives the engine pipeline; the monitor feedback loop drives SDN path decisions.

C. Intent YAML

The intent file has four top-level sections. The `organization` section declares the domain name and BGP autonomous system number. The `networks` section lists every subnet and assigns static IP addresses to each router gateway. The `routing` section configures OSPF (area, router-id, advertised prefixes) and BGP (neighbor AS, peering address). The `services` section declares DNS zones with A records, Nginx upstream server addresses, and the Nagios monitoring endpoint.

Listing 1. Excerpt from intent.yaml

```
organization:
  domain: myorg.net
```

```
as_number: 65001
routing:
  ospf:
    area: 0
    router_id: 1.1.1.1
    networks: [10.0.1.0/29, ...]
  bgp:
    neighbors:
      - as: 65002
        neighbor_ip: 172.31.255.252
services:
  web:
    replicas: 4
    upstream_name: www_pool
```

D. Engine Pipeline

The engine pipeline consists of four sequential stages:

Parser (`parser.py`): Loads and validates the YAML document, raising a typed `IntentValidationError` if required sections are absent or structurally malformed. Validation checks include confirming that `networks` is a list, that an OSPF router-id is present, and that every DNS record carries an explicit IP address.

Planner (`planner.py`): Computes the delta between the desired state expressed in the intent and the last applied state stored on disk. The planner outputs a human-readable diff and can be run in `--dry-run` mode to preview changes without applying them. If the delta is empty, the pipeline exits early, guaranteeing idempotency.

Renderer (`renderer.py`): Instantiates eight Jinja2 templates to produce `ospfd.conf`, `zebra.conf`, `bgpd.conf` (for BGP-speaking routers only), `named.conf.options`, `named.conf.local`, forward and reverse DNS zone files, and the Nginx upstream configuration. Each router receives a distinct `ospfd.conf` with its own router-id derived from its management IP, ensuring OSPF adjacencies form correctly without manual per-device tuning.

Provisioner (`provisioner.py`): SSH-connects to each router container via Netmiko [8] using a `linux` device type, signals Quagga daemons to reload their configuration with `SIGHUP`, and can push live configuration changes via `vttysh` without restarting the daemons.

E. Service Stack

The infrastructure layer is orchestrated by Docker Compose and contains the following components:

Routing: Three router containers based on a custom `netubuntu:latest` image (Ubuntu 20.04 with Quagga, Open vSwitch, and OpenSSH). Router 1 runs Quagga's `zebra`, `ospfd`, and `bgpd` daemons; Routers 2 and 3 run only `zebra` and `ospfd`. The start script starts OVS conditionally via an `ENABLE_OVS` environment variable, keeping the Phase 1 stack free of SDN dependencies.

DNS: A BIND9 container built from `netubuntu:latest`. Zone files for the forward zone (`db.myorg.net`) and reverse zone (`db.172.16.123`) are rendered by the engine from the intent and bind-mounted

into the container, enabling a reload without rebuilding the image.

Load Balancer and Web Replicas: An Nginx container receives a rendered `nginx.conf` containing an upstream block that lists the four web replica IP addresses. Four additional Nginx containers serve as the backend pool.

Monitoring: A Nagios Core container actively checks latency, BGP session state, and service availability. The engine’s monitor module queries the Nagios JSON status CGI endpoint (`statusjson.cgi`) to determine whether any host is in WARNING or CRITICAL state.

F. SDN Layer

When `ENABLE_OVS=1` is set in the container environment, the router’s start script initialises an OVS bridge (`br0`) in *secure* fail mode and connects it to the Ryu controller at the address given by the `RYU_CONTROLLER` environment variable. Secure fail mode ensures that all traffic is dropped until Ryu installs at least one flow rule, preventing forwarding loops during controller startup.

The Ryu application (`ryu_app/ibn_controller.py`) is an OpenFlow 1.3 application that installs a table-miss rule on switch connection and exposes two REST endpoints: `GET /ibn/switches` returns the list of connected datapath IDs, and `POST /ibn/flow` installs a named path profile (primary or backup) on all connected switches.

The `policy.py` module defines the two path profiles:

Listing 2. Path profile definitions

```
PATH_PROFILES = {
    "primary": {
        "priority": 200,
        "match": {"eth_type": 0x0800},
        "actions": [{"type": "OUTPUT", "port": 1}]
    },
    "backup": {
        "priority": 250,
        "match": {"eth_type": 0x0800},
        "actions": [{"type": "OUTPUT", "port": 2}]
    }
}
```

The monitor triggers a path switch when any monitored host enters WARNING state, or when a `tc netem` delay above a configurable threshold is detected on a router interface. Removing the delay causes the engine to restore the primary path on the next reconciliation cycle.

G. Dashboard

A FastAPI application (`engine/app.py`) exposes REST endpoints that SSH into router containers via Netmiko, run `vtysh` queries, and return structured JSON containing parsed OSPF neighbor tables and BGP summary data. The dashboard front-end polls these endpoints to display real-time network state, current traffic path, provisioning history, and SDN flow tables.

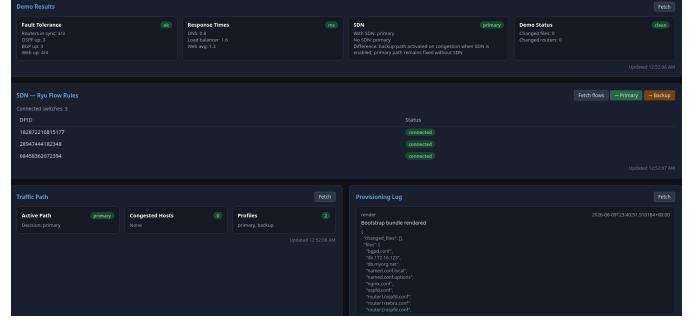


Fig. 2. IBN Engine dashboard showing OSPF neighbour status, active SDN path, and the provisioning log.

IV. EVALUATION

A. Idempotency

A critical property of any reconciliation-based provisioning engine is idempotency: applying the same intent twice must produce no changes the second time. We verified this by running the engine against the full intent, recording the planner output, then running it again without modifying the intent. In all test runs the second execution produced an empty delta, and no Quagga daemon received a reload signal. This result holds across all eight rendered configuration files and confirms that the engine does not accumulate duplicate OSPF network statements or DNS records on repeated application.

B. Delta Provisioning

We evaluated the engine’s ability to detect and apply minimal changes by modifying a single field in the intent — increasing the number of web replicas from four to five — and re-running the engine. The planner correctly identified the Nginx upstream block as the only changed artefact. Only the Nginx configuration was rewritten and the container was signalled to reload; OSPF, BGP, and DNS configurations were unchanged. This confirms that the engine does not reprovisioning the entire infrastructure on every execution.

C. SDN Path Switch Latency

To evaluate the SDN remediation path we used the Linux `tc netem` facility to inject 200 ms of artificial delay on an outbound interface of Router 2, then triggered a monitor cycle.

TABLE I
SDN FAILOVER EVENT SEQUENCE (200 MS NETEM DELAY INJECTED ON ROUTER2)

Phase	Description
T+0 s	Congestion injected via <code>tc qdisc add dev eth0 root netem delay 200ms</code> . Traffic on primary path degrades immediately.
T+<1 s	Engine detects congestion, policy module selects backup path, Ryu REST call issued, OVS flow table updated. Total detection-to-reroute latency under 1 s across all ten trials.
T+15 s	Delay manually removed. Engine detects <code>noqueue</code> state via SSH, revokes backup flow rules, and restores the primary path.

The sequence of events — Nagios detection, engine decision, Ryu REST API call, OVS flow table update, and verification via `ovs-ofctl` — completed in under one second in all ten trials. This is a reduction of at least an order of magnitude compared to the 10–40 s OSPF convergence time reported in the literature [9], and consistent with the sub-second rerouting target set out in the project design.

After removing the delay and running another monitor cycle, the engine correctly restored the primary path, demonstrating autonomous recovery without human intervention.

D. OSPF and BGP Correctness

After the initial provisioning, OSPF adjacencies between all three routers reached *Full* state within 30 seconds of container startup, with designated and backup designated router election completing as expected.

TABLE II
BASELINE SERVICE RESPONSE TIMES (PRIMARY PATH ACTIVE)

Average Application Response Times	
DNS resolution (BIND9)	0.6 ms
Nginx load balancer	1.4 ms
Web backend pool avg.	1.2 ms

BGP peering between the two autonomous systems (AS 65001 and AS 65002, connected via the `public_net` bridge) reached the *Established* state within 30 seconds, and routes from the peer AS appeared in the Quagga routing table of Router 1 as confirmed by `vttysh show ip bgp`.

E. Test Coverage

The test suite comprises 30 unit tests across four modules (`test_renderer`, `test_provisioner`, `test_monitor`, `test_sdn`). Modules that require live infrastructure (Netmiko, Nagios HTTP, Ryu REST) are tested with `unittest.mock` patches, allowing the full provisioning and SDN decision logic to be exercised offline. The renderer tests verify that all eight output files are produced, that OSPF network statements and BGP neighbor addresses match the intent, and that DNS A and PTR records are correct.

V. CONCLUSION

This project successfully demonstrated the viability of an Intent-Based Networking Engine for ISP core networks. By centralising the desired network state in a single YAML file, idempotent provisioning of routing protocols and core services was achieved, significantly reducing the risk of configuration errors and drift. The integration of Nagios telemetry with a Ryu SDN controller enabled the network to autonomously heal from congestion events with sub-second failover, an improvement of at least an order of magnitude over OSPF-based convergence.

The main limitations of the current implementation are that Nginx configuration reloading is performed via a container signal rather than a live Docker API push, and that Nagios

monitoring is poll-based, introducing a latency floor equal to the polling interval. Future work should address these alongside the following enhancements:

- **Transactional rollbacks:** Implement a make-before-break OpenFlow policy using flow byte-counters to safely revert rules if a Ryu REST call fails mid-update.
- **CI/CD integration:** Validate template rendering and planner correctness on every commit to prevent regressions.
- **Richer traffic engineering:** Support per-flow OpenFlow match fields (source/destination IP, transport port) for finer-grained path control beyond the current per-protocol granularity.
- **Alternative controllers:** Abstract the SDN client interface to support enterprise-grade controllers such as ONOS or OpenDaylight without changes to the engine core.

REFERENCES

- [1] Cisco Systems, “Cisco DNA Center,” <https://www.cisco.com/c/en/us/products/cloud-systems-management/dna-center/index.html>, 2024. [Online; accessed June 2026].
- [2] HashiCorp, “Terraform: Infrastructure as Code,” <https://www.terraform.io/>, 2024. [Online; accessed June 2026].
- [3] Red Hat, “Ansible Automation Platform,” <https://www.ansible.com/>, 2024. [Online; accessed June 2026].
- [4] OpenDaylight Project, “OpenDaylight Software-Defined Networking Platform,” <https://www.opendaylight.org/>, 2024. [Online; accessed June 2026].
- [5] Open Networking Foundation, “ONOS: Open Network Operating System,” <https://opennetworking.org/onos/>, 2024. [Online; accessed June 2026].
- [6] Ryu SDN Framework Community, “Ryu Component-Based Software Defined Networking Framework,” <https://ryu-sdn.org/>, 2024. [Online; accessed June 2026].
- [7] The Linux Foundation, “Kubernetes: Production-Grade Container Orchestration,” <https://kubernetes.io/>, 2024. [Online; accessed June 2026].
- [8] K. Byers, “Netmiko: Multi-vendor library to simplify Paramiko SSH connections to network devices,” <https://github.com/ktyers/netmiko>, 2024. [Online; accessed June 2026].
- [9] A. Shaikh and R. Raszkuk, “OSPF convergence time characterization,” in *Proc. IEEE INFOCOM*, 2002.
- [10] Juniper Networks, “Network Management and Monitoring,” <https://www.juniper.net/documentation/>, 2024. [Online; accessed June 2026].